INDEX

S.No.	ANITS Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1)	8/7	Introduction to HTML and CSS.	CO1	10	
2)	15/7	Static webpage using HTML and CSS.	CO1	10	
3)	22/7	HTML codes using audio & video tags	CO1	10	
4)	29/7	Program using javascript	CO1	10	
5)	5/8	Implement modules in NodeJS	CO2	10	
6)	12/8	Develop mini application using Express & NodeJS	CO2	10	
7)	19/8	code Tantra [static webpage]	CO1	10	
8)	2/9	code Tantra (portfolio)	CO1	10	
9)	2/9	code Tantra [simple calculator]	CO1	10	
10)	9/9	create a function & class	CO2	10	
11)	23/9	Implement http methods.	CO2	10	
12)	30/9	Develop CRUD operation	CO3	10	
				10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026	Semester:	I^st
Name of the Student:	K. HSV. Tejaswini	Roll Number:	A24126552147
Name of the Lab:	Full Stack Lab	Timings:	

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	4	4	4	1	13	10	R
8/7	2	4	3	4	2	14	10	R
15/7	3	4	4	4	1	14	10	R
22/7	4	4	4	4	1	13	10	R
29/7	5	4	4	3	2	13	10	R
5/8	6	4	3	4	2	14	10	R
12/8	7	4	4	4	2	14	10	R
19/8	8	4	4	4	2	14	10	R
2/9	9	4	4	5	2	15	10	R
9/9	10	4	4	4	2	15	10	R
23/9	11	4	4	5	2	15	10	R
30/9	12	4	3	5	2	15	10	R

Name of the Faculty: Ch. Sruthi

Faculty Signature: Ch Sruthi

Introduction to HTML and CSS?

→ HTML (Hyper text Markup language) is the staudard markup language used to create and structure content on webpages. It defines the basic structure of a webpage using elements & tags such as headings, paragraphs, links, images lists, tables forms and other multimedia content. HTML documents are interpreted by web browsers to display content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (Cascading Style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment dimensions and layouts. CSS also supports responsive design, allowing webpages to adopt to different screen sizes and devices. CSS can be applied using inline, internal and external stylesheets.

→ HTML and CSS work together to develop web-pages where HTML provides the structure and content, while CSS provides the styling. layouts, visual presentation the combination of both forms the foundation of modern web development and enables the creation of organized attractive and user friendly web interfaces.

Difference between MERN stack and Full Stack >

→ MERN Stack is a Javascript based technology stack used for developing modern web applications MERN stands for MangoDB, Express.js, React.js, Node.js. MangoDB is used for database management, Express.js handles the backend framework and APIs; React.js is used to develop frontend user interface and Node.js provides the server-side runtime environment.

→ Full-Stack Development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementation server-side logic managing databases.

HTML components:

1. HTML tags - Define the structure and elements of a webpage.
2. Headings - used to Display titles and headings using <h1> to <h6>
3. Paragraphs - used to display text using <p> tag
4. Links - used to navigate between webpages using <a> tag.
5. Images - used to display images using <image> tag.
6. Lists - used to create ordered and unordered lists using <ol>, <ul>, <li>.
7. Tablets - used to organize data in columns & rows.
8. Forms - used to collect user input using elements such as text boxes, buttons and checkboxes.
9. Div & Span - used to group and organize webpage content.
10. Semantic Elements: Elements such as <header>, <nav>, <section>, <article> <footer>

Provide meaningful webpage structure.

CSS components:

1. Selectors - Select HTML elements to apply styles
2. Properties - Define the visual characteristic of elements.
3. Values - Specify the setting of a css property
4. Colors - Define text, background, border colors.
5. Fonts - control font family, size, style and weight.
6. Backgrounds - Set Background colors and images.
7. Borders - Define the border style, width and radius.
8. Margin - controls the space outside as element.
9. Padding - controls the space inside element
10. Layout - controls the arrangement of elements using flexbox, grid and positioning

HTML VS HTML5

HTML	HTML5
older version of HTML	Latest major version of HTML
limited multimedia support	Supports audio & video
uses external plugging for multimedia	No External plugs needed
Has fewer semantic tags	Provides semantic tags.
No <canvas> element.	Supports <canvas>
limited web storage	supports local storage
complex DOCTYPE declaration	simple <!DOCTYPE HTML>

CSS Vs CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic styling features.	Provides advanced styling features.
Limited animations	supports animations & transitions.
No flexible layout systems. limited visual effects	Supports flexbox & grid supports, shadows, gradients, effects.
No media queries	Supports media queries.
uses a single specification	Divided into different modules.

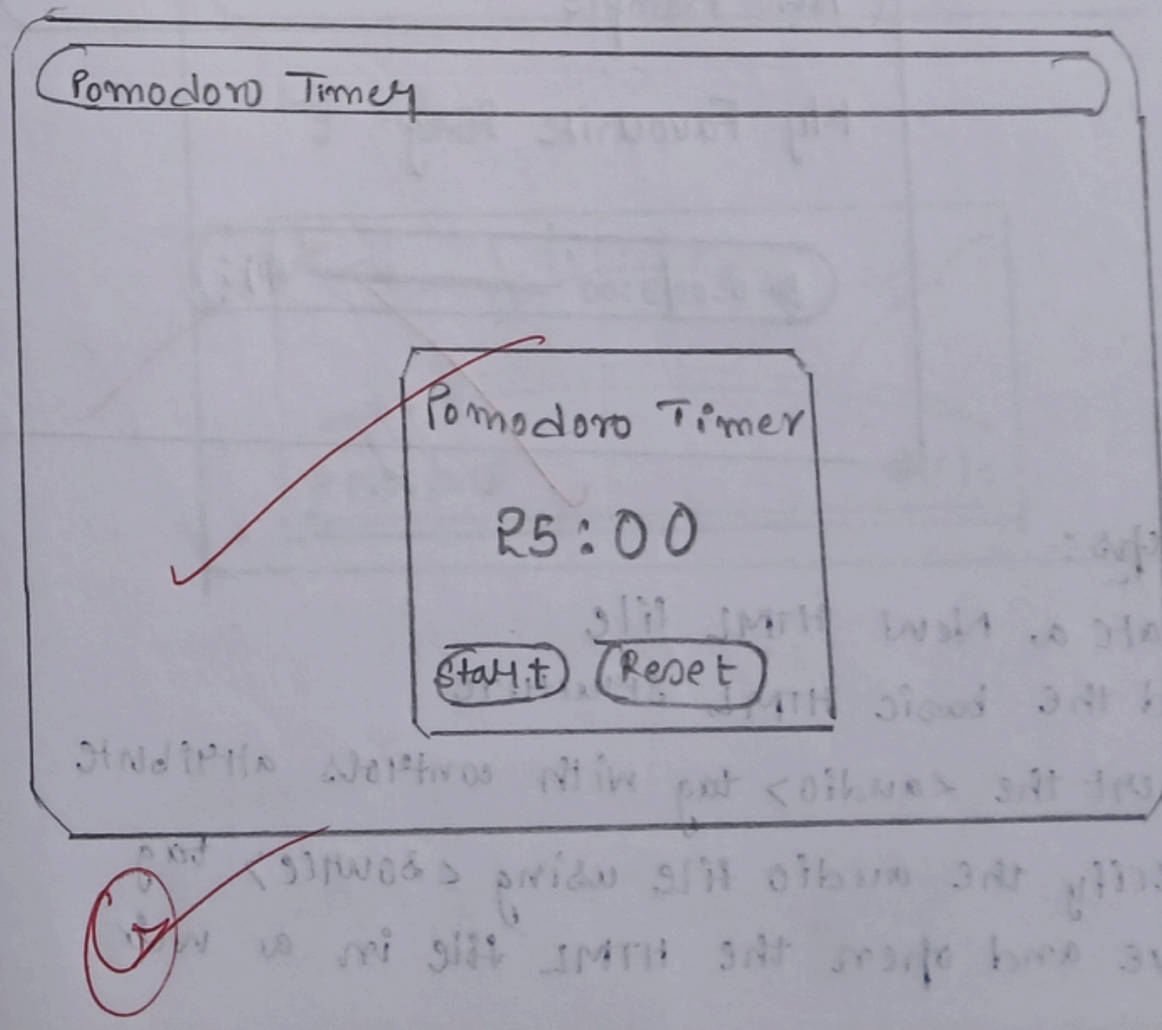
15/7/26

Design a static webpage using HTML & CSS.

Pomodoro Timer, using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
      body {
        font-family: Arial;
        background: #f5f5f5;
        text-align: centre;
        Padding: 50px;
      }
    timer{
        width: 300px;
        margin: auto;
        Padding: 30px;
        background: white;
        border-radius: 15px;
        box-shadow: 0 0 10px gray;
      }
    h1{
      color: #e74c3c;
    }
    time {
      font-size: 50px;
      margin: 20px;
    }
    button{
    Padding: 10px 20px;
    margin: 5px;
    border: none;
    border-radius: 5px;
    background: #e74c3c;
    color: white;
    }
</style>
</head>
</body>
    <div class = "Timer">
      <h1> Pomodoro Timer </h1>
      <div class = "time" > 25:00 </div>
      <button> start </button>
      <button> Reset </button>
    </div>
    </body>
    </html>
```

output:



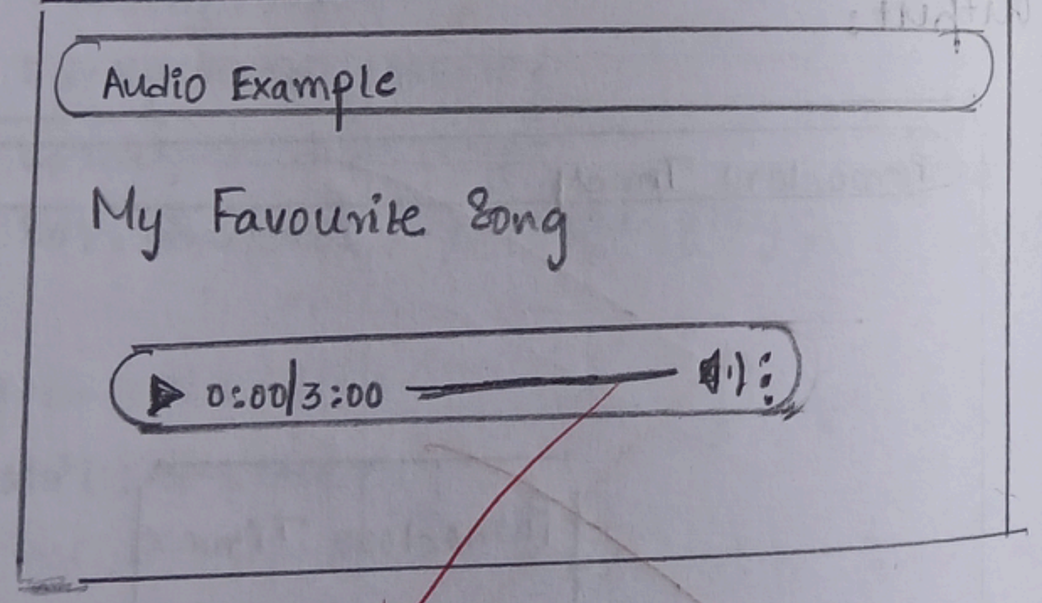
22/7/26

i) Design an HTML code using audio tags

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favourite Song </h2>
    <audio controls>
        <source src = "song.mp3" type = "audio/mpeg">
        your browser does not support the
        audio tags
    </audio>
</body>
</html>
```

Output:



Steps:-

create a New HTML file

Add the basic HTML structure

Insert the <audio> tag with controls attribute

specify the audio file using <source> tag

save and open the HTML file in a web browser

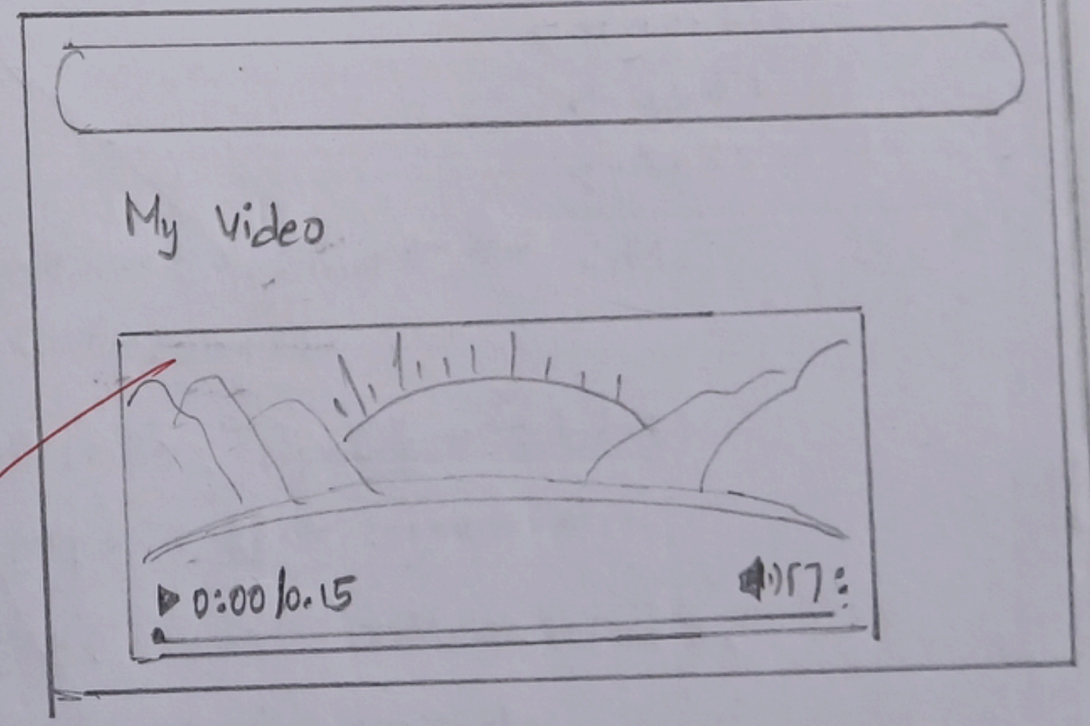
Play the audio using displayed controls.

ii) Design a HTML code using video tags.

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> My video </h2>
    <video width = "400" controls>
        <source src = "video.mp4" type =
                                "video/mp4">
    </video>
</body>
</html>
```

Output:



Steps:
Create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with controls attribute
Specify the video file using <source> tag
Save the file and open it a web browser
Play the video using displayed controls.

Write a Program using JavaScript (arrays, functions)

code:

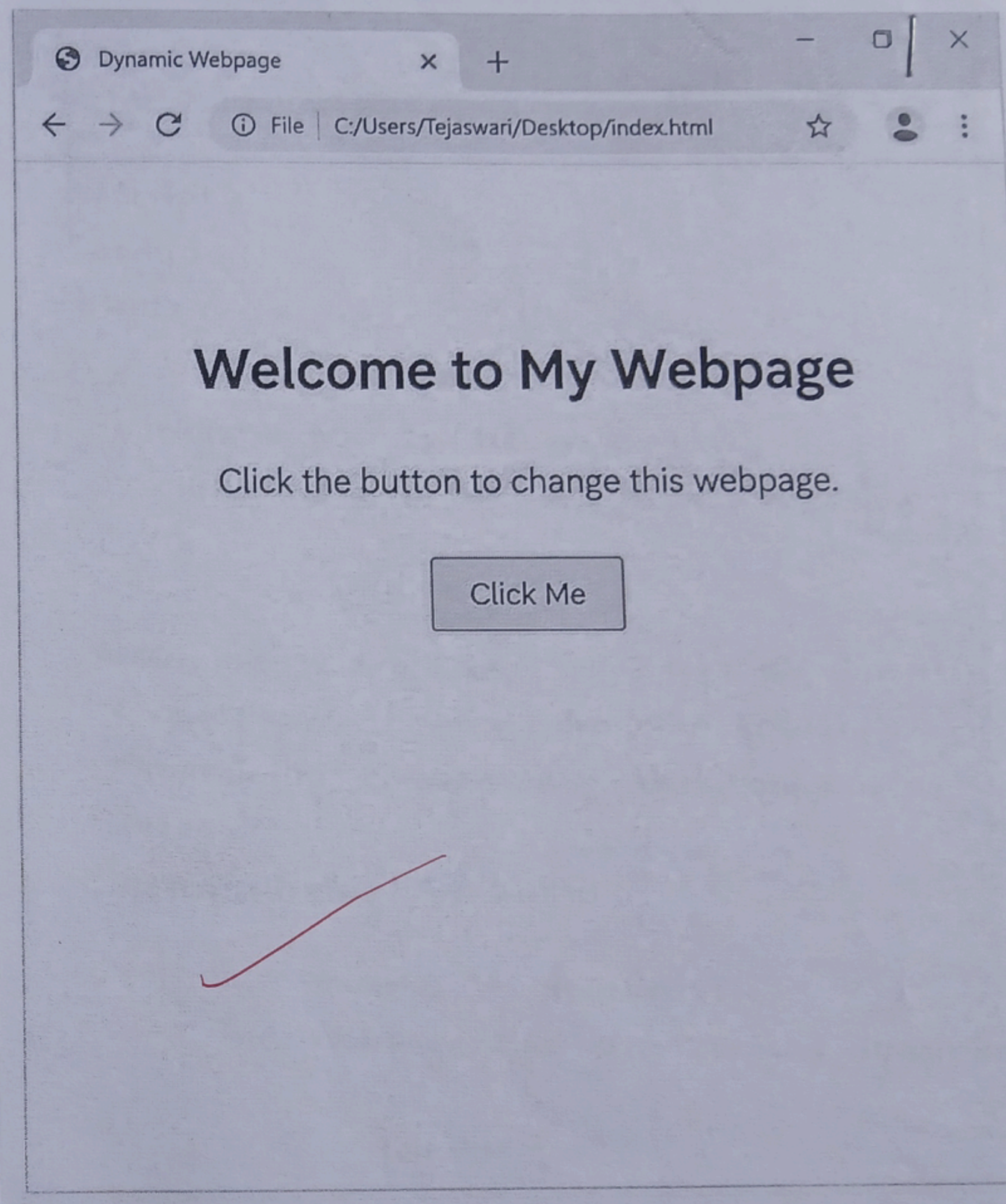
```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];
function displayArray(arr) {
    console.log("Array Elements : ");
    for (let i=0; i<arr.length; i++) {
        console.log(arr[i]);
    }
}
function findSum(arr) {
    let sum=0;
    for (let i=0; i<arr.length; i++) {
        sum += arr[i];
    }
    return sum;
}
```

EXPERIMENT-04

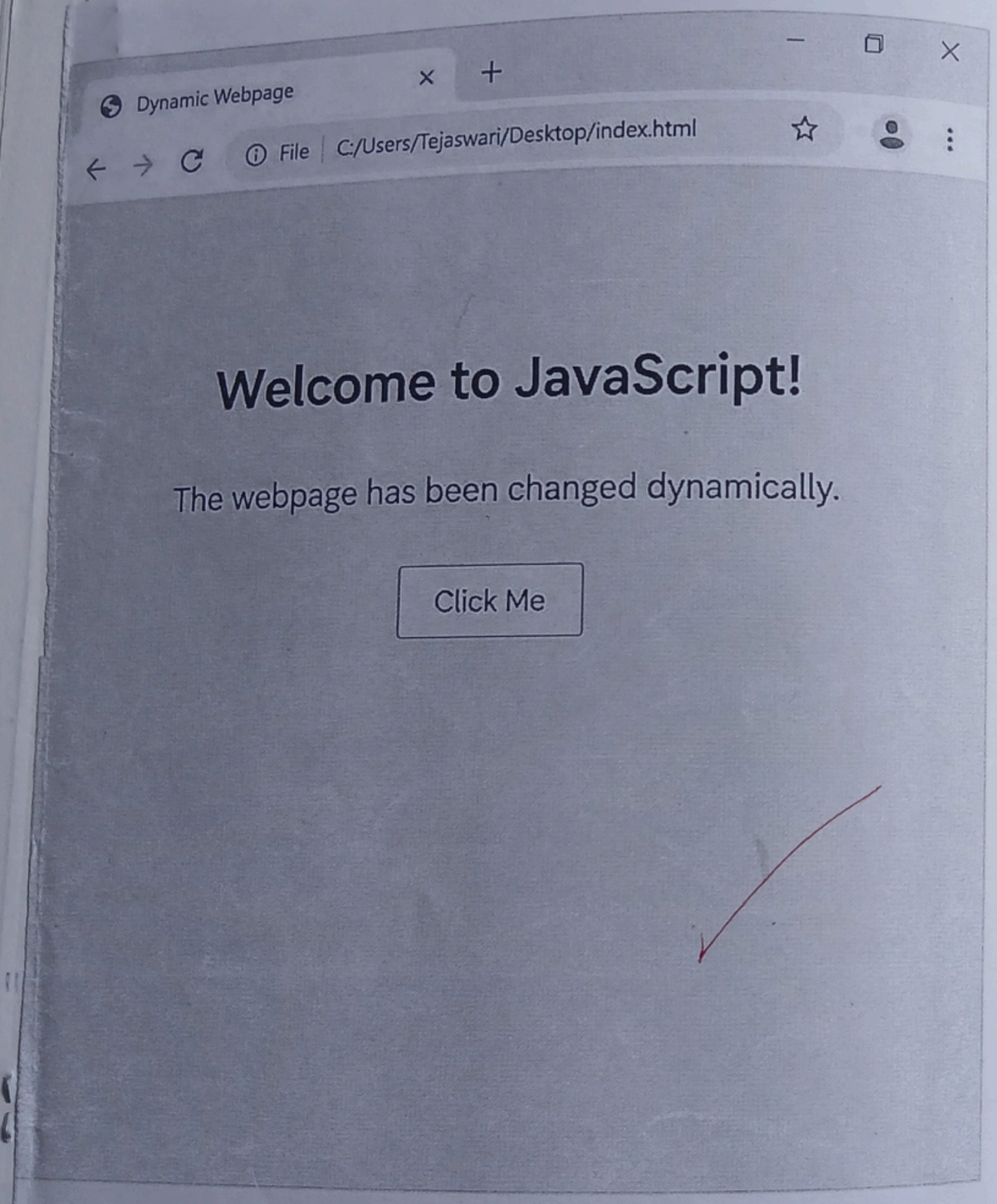
4) Develop dynamic webpage using Java Script

Program:

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic webpage </title>
    <style>
        body {
            text-align : center;
            font-family : Arial;
            margin-top : 100px;
        }
        button {
            padding : 10 x 20px;
            font-size : 16px;
            cursor : pointer;
        }
    </style>
</head>
<body>
    <h1 id="heading"> Welcome to
        My Webpage </h1>
    <p id = "message"> click the button
        to change this webpage </p>
    <button onclick = "change page()">
        clickMe </button>
    <script>
        function changepage(){
        document.getElementById ("heading")
        "innerHTML = " Welcome to Javascript !";
        document.getElementById ("heading")
```



Before clicking the button



After clicking the button

EXPERIMENT-05

Implement Modules in Node JS.

In Node.js you can implement modules using distinct systems. commonJS(CJS). which is traditional default, and ES modules (ESM), the modern javascript standard

1. The commonJS Method (default):

commonJS uses module exports to share code and square() to import it. This system loads modules synchronously

a. create the module (math.js):

```
const add = (a,b) => (a+b);
const subtract = (a,b) => a-b;
module.exports = {
        add,
        subtract
};
```

b. use the module (app.js):

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

o/p: 15

5

2. The ES modules Method (modern standard):

ES Modules use export to share code and import to use it. To make Node.js recognize ES modules, you must either use the .mjs file extension or add "type" "module" to your package.json on file

a) create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "supercalc";
export default calculatorName;
```

b) use the module (main.js)

```
import calculatorName, {multiply, divide}
        from './calculator.js';
console.log(calculatorName)
console.log(multiply(4,2));
console.log(divide(4,2));
```

o/p:- Supercalc
8
2

3) using Built-in core modules:

Node.js also comes equipped with dozen of built-in core modules (like fs for file system or path for file routing) that don't need installation

we will load them by module names:

```
// common JS style
const fs = require('fs');
// ES Modules style
import fs from 'fs';
```

Here are some of the most commonly used ones:

1. fs = file system operation
2. http = HTTP server and client
3. path = File path utilities.
4. os = operating system utilities
5. Events - Event handling
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. query string - URL query string handling

Eg:- Simple HTTP Server

```
http.createServer((req,res) => {
  res.writeHead(200, {'content-Type':
                      'text/html'})
  res.end('Hello World!');
}).listen(808);
```

o/p: Hello World!

EXPERIMENT-06

6. Develop mini application using express and Node JS simple mini application using node JS + Express JS.

Student Maugement System

1. Add a student
2. View all students.
3. delete a student
4. uses Express js routes.
5. Simple HTML interface

Project structure:

```
student.app/
    ├── server.js
    └── public
        └── index.html
```

Server.js:

```
const express = require("express");
const app = express()
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
        res.json(students);
});
// Add
app post("/students", (req, res) => {
        const student = {
```

```
    id = students.length+1;
    name = req.body.name;
    course = req.body.course;
  };
  students.push(student);
res.json({message: "student added successfully",
          student: student});
});
//Delete
app.delete("/students/:id", (req,res) => {
const id = parseInt(req.params.id);
students = students.filter(student =>
    student.id !== id);
res.json({
  message: "student deleted successfully"});
});
//start server
app.listen(PORT,() => {
  console.log('server running at http://localhost:
              ${port}');
});
```

public/index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Management</title>
</head>
<body>
  <h1> Student Management System </title>
  <input type="text" id="name"
  placeholder="Enter student name">
  <input type="text" id="course"
  placeholder="Enter course">
  <button onclick="addStudent()">Add Student
                                  </button>
  <h2> Student list </h2>
  <ul id="studentlist"></ul>
  <script>
    function addStudent(){
      let name = document.getElementById("name").
                                  value;
      let course = document.getElementById("course"
                                  ).value;
    fetch("/students",{
        method:"POST",
        headers:{
          "Content-Type":"application/json"},
        body: JSON.stringify({
            name:name,
            course:course
          })
      })
      .then(response => response.json())
      .then(data => {
          alert(data.message);
document.getElementById("name").value="";
document.getElementById("course").value="";
    display.Students();
    });
}
```

```
function displaystudents(){
  fetch("/students").then(response => response.
                                        json())
  .then(students => {
    let list = document.getElementById("studentlist
    list.innerHTML = "";
    students.forEach(student => { let li =
    document.createElement("li");
      li.innerHTML = student.id + "-" +
                     student.name + "-" +
                     student.course + "<button
    onclick='deletestudent(" + student.id + ")'>Delete
    </button>";
    list.appendchild(li)
          })
        });
    }
    function deleteStudent(id){
      fetch("/students/" +id, {method
                          "DELETE"})
    .then(response => response.json())
    .then(data => {
          alert(data.message)
          displaystudents();
        });
    }
    displaystudents();
  </script>
</body>
<html>
```

output:

Student Management system

Enter student name

Enter course

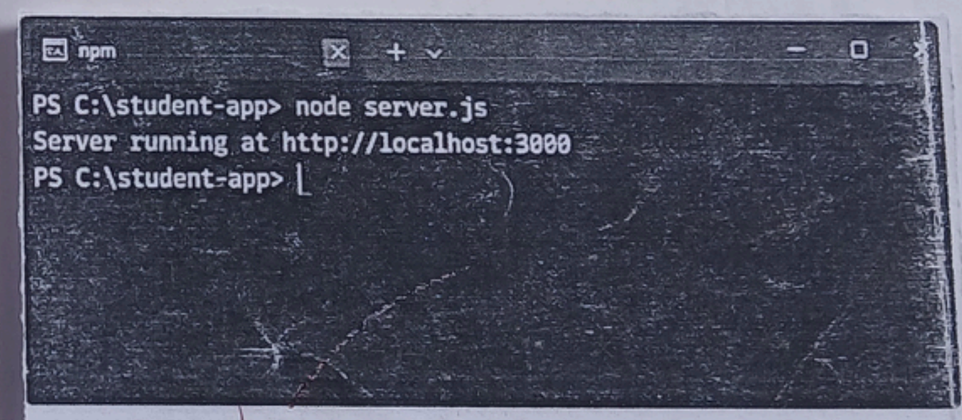
Add student

Student list :

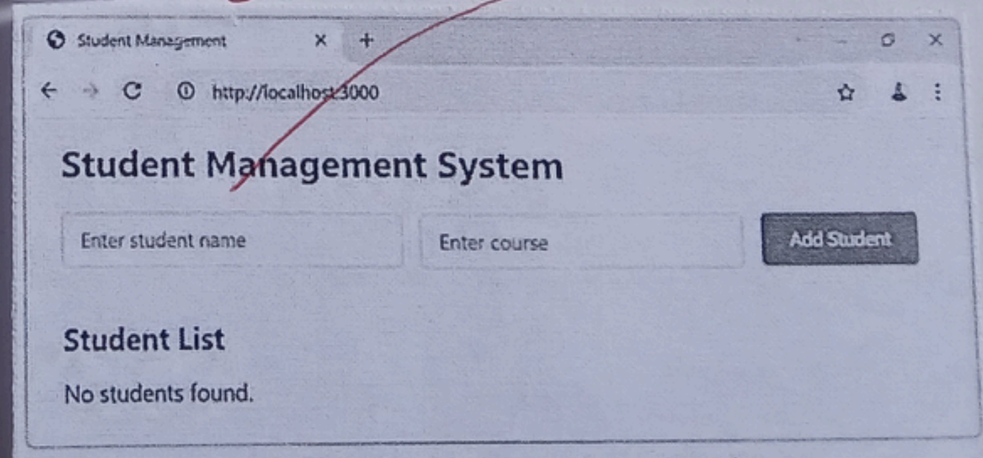
1. Tejaswini. CSM delete
2. Sita - CSE delete.

- the Express.js server is successfully started at http://localhost:3000

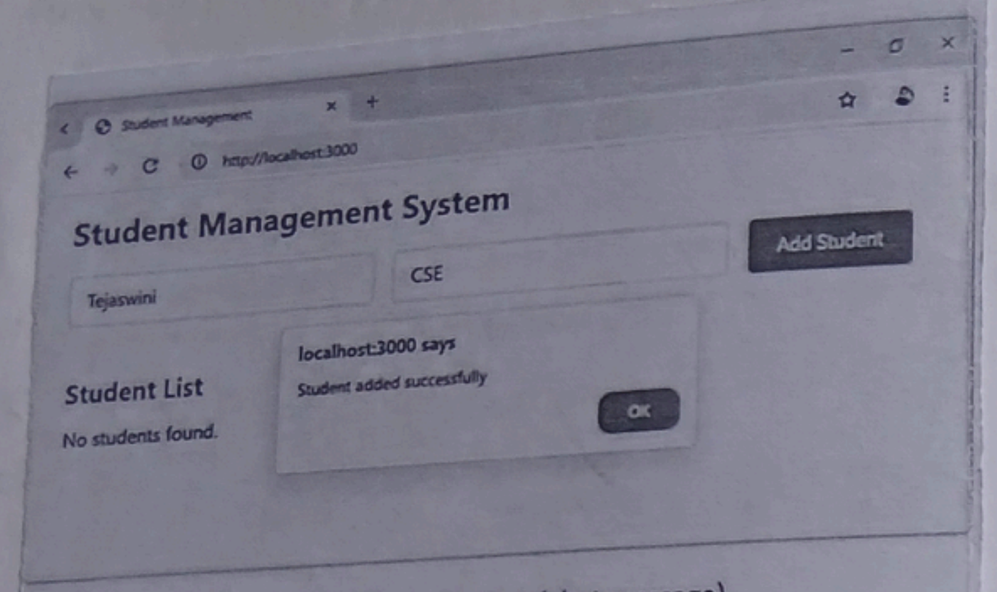
the browser displays the student Management system with fields.



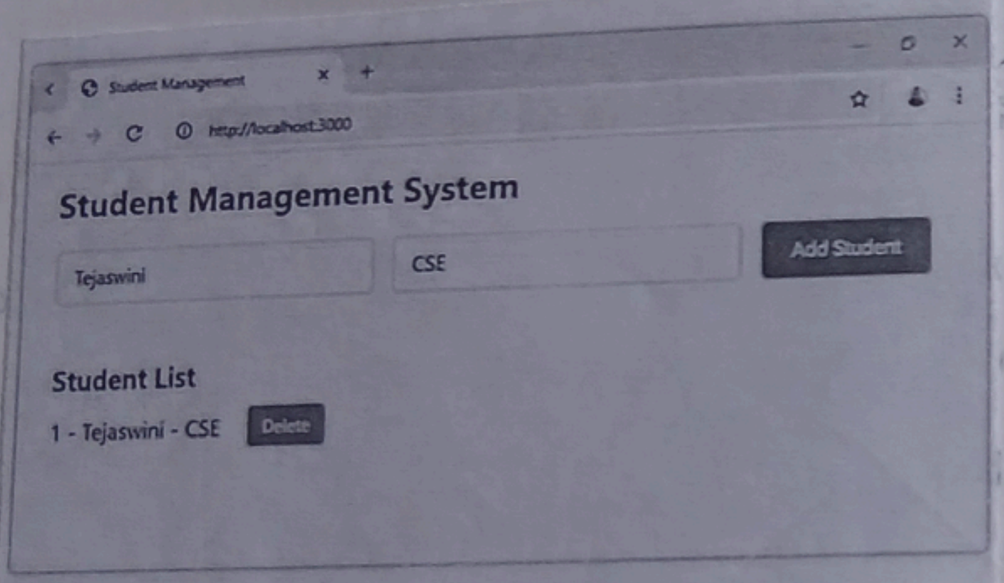
1. Terminal Output (after running `node server.js`)



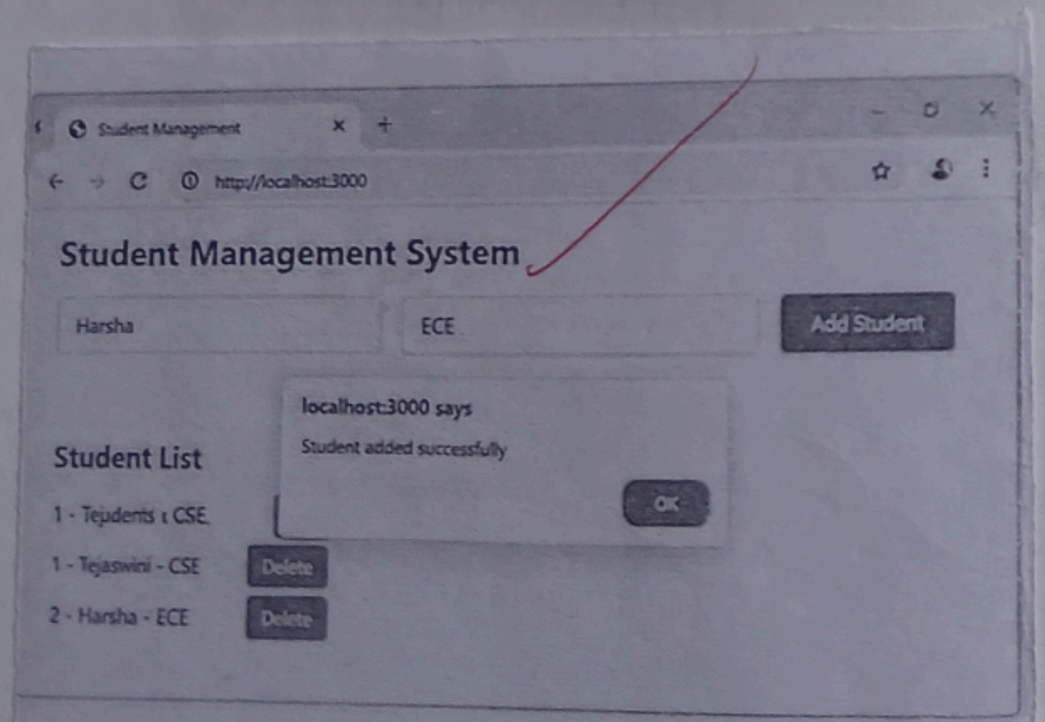
2. Initial Output in Browser (no students)



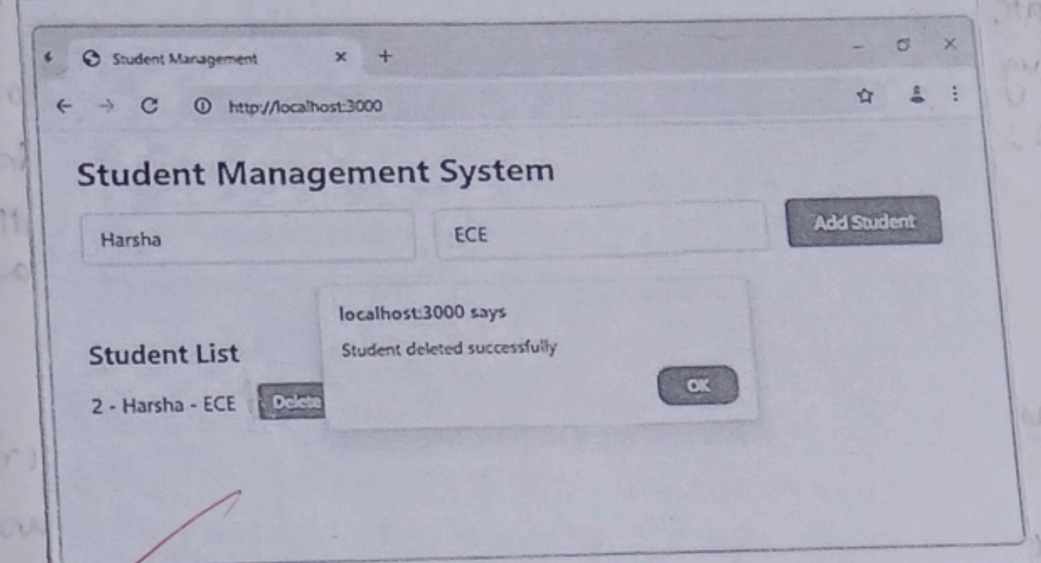
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

EXPERIMENT-07

7) create a Functional component and class component using react

Aim: To develop a react application using Functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

- Functional component:

A functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as usestate to manage state and update the webpage dynamically

Program:

```
import React, {useState} from
'react';
function studentFunctional(){
    const [name, setName] = usestate("Tejas wini");
    const [course, setCourse] = UseState("CSM");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Teju");
    };
const changeCourse = () => {
    setCourse("CSD")
};
const incrementCount = () => {
    setCount(count+1):
  };
return(
 <div style ={{
    border: "2px solid blue"
    Padding : "20px",
    margin : "20px",
    borderRadius : "10px",
    textAlign : "center"
}}>
<h1> Student Details </h1>
<h2> Functional component </h2>
<p> <b> Name : </b> {name} </p>
<p> <b> course : </b> {course} </p>
<p><b> Button clicks : </b> {count} </p>
<button onClick ={changeName}>
        chaugeName </button>
<button onClick = {changecourse}>change
        course </button>
<button onClick = {incrementcount}> Increment
        </button>
  </div>
};
```

Output:

when the user clicks the buttons, the name course and count chauge dynamically without reloading the webpage

• class component

A class component is an ES6 Javascript class that extends React component. It uses the render() method to return JSX and manages state using this state and this setState()

Program:

```
import React. {Component} from
'react':
class StudentClass extends Component {
    constructor(props). {
        super(props);
        this. state = {
            name = "Tejaswini",
            course = "computer science"
            count : 0
        };
    }.
    changeName = () => {
        this. setState ({
            name = "Teju"
        });
    };
    changeCourse = () => {
        this. setState ({
            course : "CSM"
        });
    },
    incrementCount = () => {
        this. setState ((prevstate) = ({
            count = prevstate. count + 1
        }));
    };
    render() {
        return (
            <div style = {{
```

```
        border = "4px solid green",
        padding = "20px",
        margin = "20px",
        borderRadius = "10px",
        textAlign = "centre"
    }}>
    <h1> Student Details </h1>
    <h2> class component </h2>
    <p><b> Name : </b> {this.State.name} </p>
    <p><b> course : </b> {this.State.course} </p>
    <p><b> Button clicks : <b> {this.state.count}
                                    </p>
<p><b> Name : </b> {this.sta
<button onclick = {this.changeName}>
                    change Name </button>
<button onclick = {this.changecourse}>
                    change course </button>
<button onclick = {this.incrementCount}>
                    Increment </button>
    </div>
```

output:

EXPERIMENT-08

8, Implement http methods using JSON server

Aim: To implement http methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (create, Read, update and Delete) operations on student results

Theory: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a server

- GET - Retrives data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - updates selected fields of a record
- DELETE - Removes a record from the server

a. create a JSON server:

Step 1: Initialize the project

open the vs code terminal and execute the following commands.

```
mkdir JSONserver
cd JSONserver
npm init-V
npm install json server
```

step-2: create db.json

create a file named db.json and add the following student data.

```
{ "students" = [
  {
    "id" = "1",
    "name": "Tejaswini",
    "course" = "CSM",
    "marks" : 90
  },
  { "id" = "2",
    "name" = "chaitrika"
    "course" : "CSM",
    "marks" : 90
  },
  { "id" : "3"
    "name" : "Sravani"
    "course" : "IT",
    "marks" : 88
  }
```

Step-3: Start the JSON server

for the current JSON server version, execute:

npx json-server db.json

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students.

2. Implementation of HTTP methods :

a. GET Method : The GET method is used to retrieve student records from JSON server

code :

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
        console.log(data);
    })
.catch(error => {
        console.log(error);
});
```

output:

B. POST Method : The post method is used to insert a new student record into the server

code:

```
fetch("http://localhost:3000/students', {
  - method: 'POST',
    headers = {
     'content-Type': 'application/json'
  },
    body : JSON.stringify({
        name : 'Rahul',
        course : 'ISE',
        marks : 92
    })
})
.then(response => respone.json())
.then(data => {
        console.log(data);
    })
.catch(error => {
        console.log(error);
    })
```

output:

C. PUT Method : The put method is used to replace an existing student record with updated data

code:

```
fetch('http://localhost:3000/student/1', {
  method: 'PUT',
    headers : {
      'content-Type': 'application/json'
  },
    body : JSON.stringify({
        name : 'Tejaswini',
        course : 'AIML',
        mark : 95
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
```

```
        console.log(error);
    });
```

output:

D. PATCH Method: The PATCH method is used to update only selected fields of an existing student record.

```
code: fetch('http://localhost:3000/students/2', {
        method: 'PATCH'
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({
            marks: 96.
        })
    })
    .then(response => response.json())
    .then(data => {
    console.log(data);
})
    .catch(error => {
        console.log(error);
    });
```

output:

E. DELETE Method: The DELETE method is used to remove an existing student record from the server

```
code: fetch('http://localhost:3000/students/3', {
        method: 'DELETE'
    })
    .then(response => {
        if(response.ok){
            console.log("Student deleted
                        successfully");
        }
    })
    .catch(error => {
        console.log(error)
    });
```

output:

- After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining student will be:

output:

Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON server the CRUD operations were performed on student records. the data was retrieved, inserted updated and deleted using HTTP requests.

EXPERIMENT-09

9. Develop CRUD operations using MongoDB

Aim: To develop a Web application using Node.js, Express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records

Theory:

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a database

Mongoose is an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations

1. Installation and project setup:

```
mkdir MongoDB.CRUD
cd MongoDB.CRUD
npm init-y
npm install express mongoose
```

Program:-

server.js

```
const express.require('express');
const mongoose = require ('mongoose');
const app = express();
const PORT = 3000;
app.use (express.json());
mongoose.connect ('mongodb://127:0.0.1:
                   27017/college)
.then() => console.log ("MongoDB connected"))
.catch (err => console.log(err));
```

```
//create student
const studentSchema = new
mongoose.Schema({
    name: {
        type: String
        required: true
    },
    course: {
        type: String,
        required: true
    },
    marks: {
        type: Number
        required: true
    }
});
//create student model
const student;
    mongoose.model('Student', studentSchema);
//CREATE Add
app.post('/students', async(req, res) => {
    try{
    const student = new student(req.body);
    const saved student = await student
                                .save();
    res.status(201).json(savedStudent);
    }catch(error){
        res.status(400).json({message:
                        error.message});
    }
});
// READ. Get all student
```

```
student findByIdAndUpdate(
    req.params.id
    req.body
    {new:true, runValidators:true}

});
```

```
if(!student) {
    return res.status(404).json({
      message: "student not found"
    });
}
res.status(200).json(student);
}catch(error){
    res.status(400).json({message
      error.message});
}
});
//DELETE
app.delete('/students/:id', async
                    (req,res) => {
  try{
  const student = await Student.
      findByIdandDelete(
        req.params.id
      );
if(!student){
  return res.status(404).json({
    message: "Student not found"
  });
}
res.status(200).json({
message: "Student deleted
                    successfully"
});
}catch(error){
  res.status(400).json({message:
                    error.message});
```

output:-